

Windows Build Status

What is proven to work on native Windows right now, what still needs a real machine to confirm, and the exact steps to build and run it.

01 Status

BUILD

Verified

`forward_search/`'s C++/OpenCL code compiles and links on native Windows via MinGW-w64 (MSYS2), confirmed by GitHub Actions.

RUN

Not yet verified

GitHub's Windows runners do not reliably have a usable OpenCL device, so CI only proves the build compiles and links, not that it runs correctly against real data.

- ① The convenient single command workflow (`projection-center pipeline --backend cpp, run_all_128.sh`, and so on) has not been tested on Windows. Only `forward_search/`'s own C++ build was attempted this session. `projection-center` is a separate Python package (with a `pyopencl` dependency) that has not been touched for Windows at all.

02 A real blocker found and fixed

CppBackend's binary path helpers used to always resolve the extension less Linux names (`forward_search`, `forward_search_resample`), so on Windows

`Path(...).exists()` would always be `False` and `--backend cpp` would report "binary not found" even after a successful build. Fixed to resolve `forward_search.exe` / `forward_search_resample.exe` on Windows instead.

CI now has a second job that installs `projection-center` via a native Windows Python and attempts the full pipeline `--backend cpp` (search and resample, one command) against a tiny synthetic dataset. That job has not run yet as of this writing, so whether the full pipeline actually completes on Windows is still an open question, just no longer blocked by this specific bug.

Without `projection-center` confirmed working, running `forward_search.exe` and `forward_search_resample.exe` directly still hits the pose format mismatch already documented for Linux: the raw `forward_search.exe --output` always writes an HDF5 pose file, but the raw `forward_search_resample.exe --pose` always reads JSON. They cannot be chained by hand on any platform, the JSON conversion only happens inside `projection-center`'s own CLI.

03 Building

Using MSYS2's MinGW64 environment, the same toolchain the CI workflow uses.

```
# from an MSYS2 MINGW64 shell
pacman -S --needed mingw-w64-x86_64-toolchain mingw-w64-x86_64-meson \
  mingw-w64-x86_64-ninja mingw-w64-x86_64-hdf5 mingw-w64-x86_64-boost \
  mingw-w64-x86_64-ocl-headers mingw-w64-x86_64-ocl-icd

# MSYS2's ocl-headers package does not include the C++ bindings header
# forward_search/ uses, fetch it once, matching the CI workflow:
curl -sSL -o /mingw64/include/CL/ocl.hpp \
  https://raw.githubusercontent.com/KhronosGroup/OpenCL-CLHPP/main/include/CL/ocl.hpp

cd forward_search
meson setup builddir
meson compile -C builddir
```

Produces `builddir/forward_search.exe` and `builddir/forward_search_resample.exe`, same as `forward_search` and `forward_search_resample` on Linux.

04 Running (search only)

Same flags as Linux, per `RUNNING.md`. Command syntax verified, actual execution against a real OpenCL device not yet confirmed.

```
./builddir/forward_search.exe --data path\to\data.hdf5 --mode buffer \
  --kernel kernels/forward_search.cl --output pose.h5
```

Prints the same `MSE: / xshift: / alpha: / beta:` summary as the Linux binary, if an OpenCL device is actually present and working on the machine, that part has not been confirmed yet.

05 Local Windows testing options considered

Looked into running or testing this on this Linux dev machine instead of waiting on CI each time. Three options, all hit the same wall.

QEMU (already installed on this machine)

Full hardware virtualization, can boot an actual Windows OS, real kernel, real driver stack. Needs a Windows ISO and license, and without GPU passthrough configured (nontrivial setup), the guest still would not have real OpenCL, the same limitation as GitHub's own runners.

Wine

Not a VM, a compatibility layer that runs Windows .exe files directly on Linux by reimplementing the Windows API. Not installed here. Could plausibly run the already built `forward_search.exe` directly on this machine for a quick smoke test (argument parsing, HDF5 loading, does it get as far as OpenCL init), but OpenCL support under Wine is unreliable, real GPU search very likely would not work through it.

GitHub Actions windows latest (what is actually being used)

Zero local setup, genuinely native Windows, already working.

- ① All three hit the same limitation: no real GPU or OpenCL device without an actual physical Windows machine. None of the local options gets meaningfully further than CI already does, they would mainly help poke at a failure interactively rather than wait for a run. Sticking with CI as the primary signal for that reason.

06 What is still open

- Actually running (not just building) on a real Windows machine with a GPU, to confirm the OpenCL kernel executes correctly there.
- Confirming `projection-center pipeline --backend cpp` actually completes on Windows: the binary path bug that would have blocked it outright is fixed, and CI now attempts it against a synthetic dataset, but that job has not run yet, so this is fixed but unconfirmed, not done.
- `run_all_128.sh` / `run_all_512.sh` themselves have not been checked for Windows compatibility yet. Both call `python3` four times each, a native Windows Python install typically only provides `python`, not a `python3` alias, so these scripts would likely need that fixed, and would need Git Bash or MSYS2 bash to run their bash syntax at all.
- This was deliberately done via MinGW-w64, not MSVC. If MSVC is a hard requirement, the flag fixes made for MinGW-w64 (portable PI constant, `boost_json` library name resolution) do not fully cover it. MSVC also needs `/O2` instead of `-O3` and different linker syntax throughout `backend.py`'s pybind JIT build, which has not been attempted.